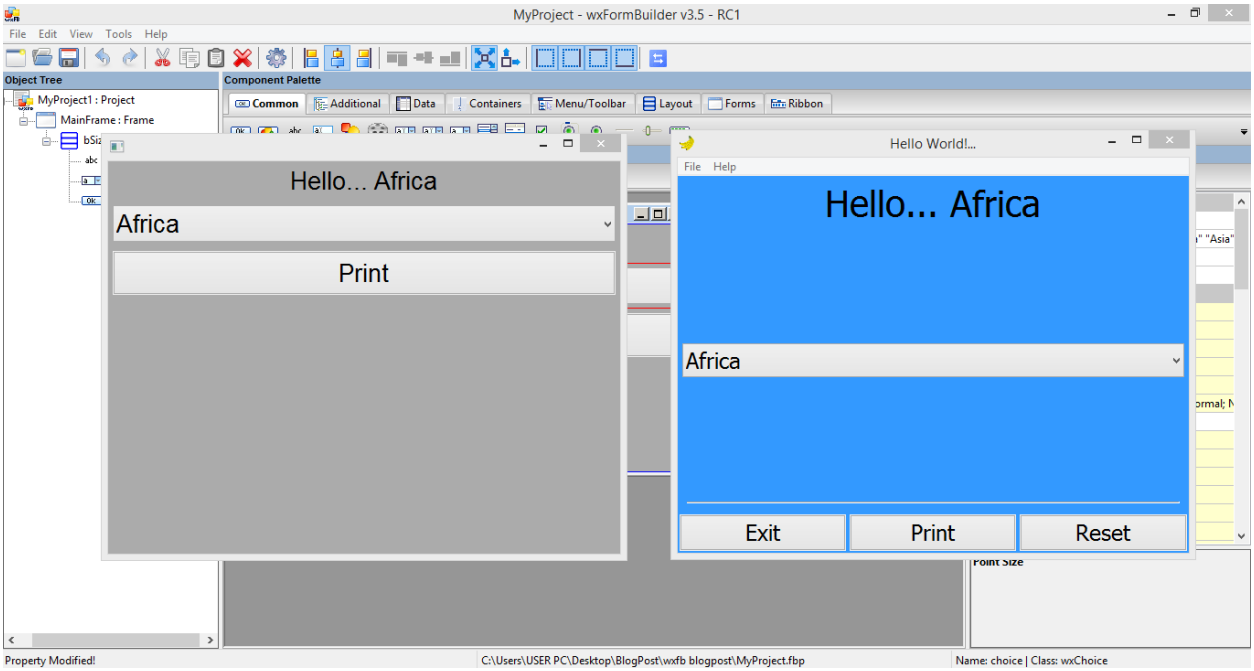
wxFormBuilder - Tutorial on "A GUI for making GUIs" for Python

By

Website: www.UmarYusuf.com/wxpy
Email: umaryusuf49@gmail.com



Graphical User Interfaces (GUIs) in Python are commonly created using Tk via the Tkinter package. However, at the moment of writing this post, designing GUI via the Tkinter package is done purely in code (python syntax) which means that a simple dialog window may consist of approximately 100+ lines of code.

Wouldn’t it be pleasant if we had a visual tool for making GUIs? That is “A GUI for making GUIs”. Creating GUI with code (WxPython) is too tedious work and it requires lots of attention and time. With WXFormBuilder, you create GUI much faster and efficiently in less time. In most cases this is faster than writing this code by hand.

That is what ‘wxFormBuilder’ is set to realize. WxFormBuilder is a Rapid Application Development (RAD) tool for wxWidgets GUI design. It is an open source GUI designer application for wxWidgets toolkit, which allows creating cross-platform applications.

WxFormBuilder allows you to design your GUIs visually and save them in a wxFormBuilder Project File - *.fbp file (which is just an XML file listing every component in your GUI and defining each component’s properties). Then, the *.fbp file can be converted automatically into a Python *.py file ready to be imported into your Python program. It also serves as Source code generation

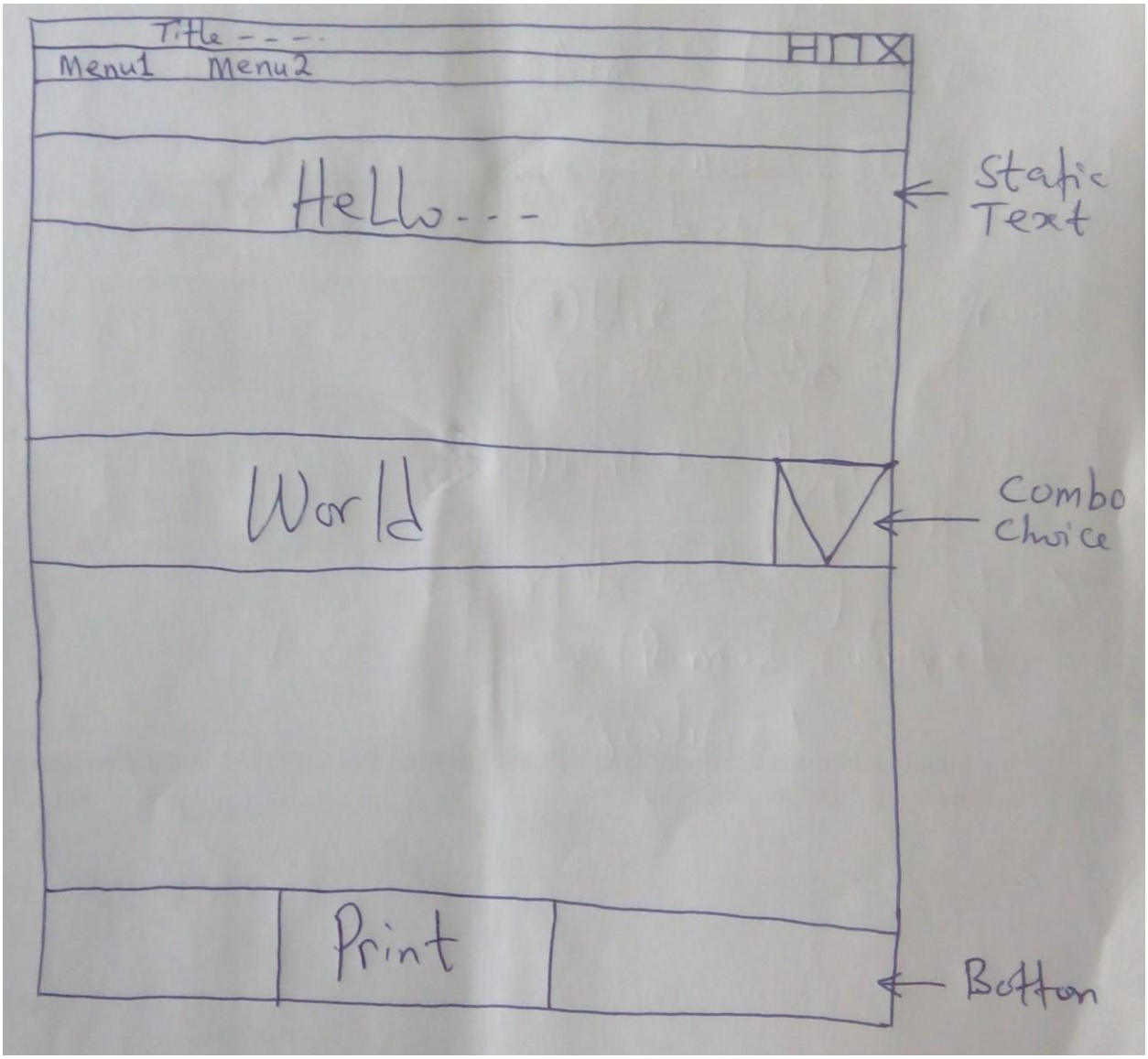
other programming languages are supported such as: C++, PHP, Lua and XRC.

Tutorial objective:

I will guide you through the basics of wxFormBuilder as used with wxPython generated code. The goal is for you to build a usable First GUI in Python.

We are going to create a little application that will take **selected** text from the user *"choice box"* and append/print the chosen text to a **static text box** that has "Hello…" on it.

The end result will be a GUI (as sketched below) that just outputs: Hello… World, or Hello… Africa, or Hello… Asia, or Hello… Europe, or Hello… America - on the displayed static text box.


Title - - -
HTTX
Menu1 Menu2
Hello - - -
Static Text
World
Combo Choice
Print
Button

Prerequisites:

We need to have the following packages installed:-

~ Python 2.7

~ wxpython 3.0.2.0

~ wxformbuilder 3.5-RC1

~ Pyinstaller to freeze the application (create an .exe on Windows or a .app on osx)

~ Text editor (Sublime Text, Notepad++, Pycharm or anything similar).

Combination of any versions will do as long as they are compatible with each other. I used Python 2.7.10 Anaconda 2.3.0 (64-bit), wxPython 3.0.2.0 (msw-unicode), wxFormBuilder 3.5-RC1 (unicode) and Sublime Text 3.

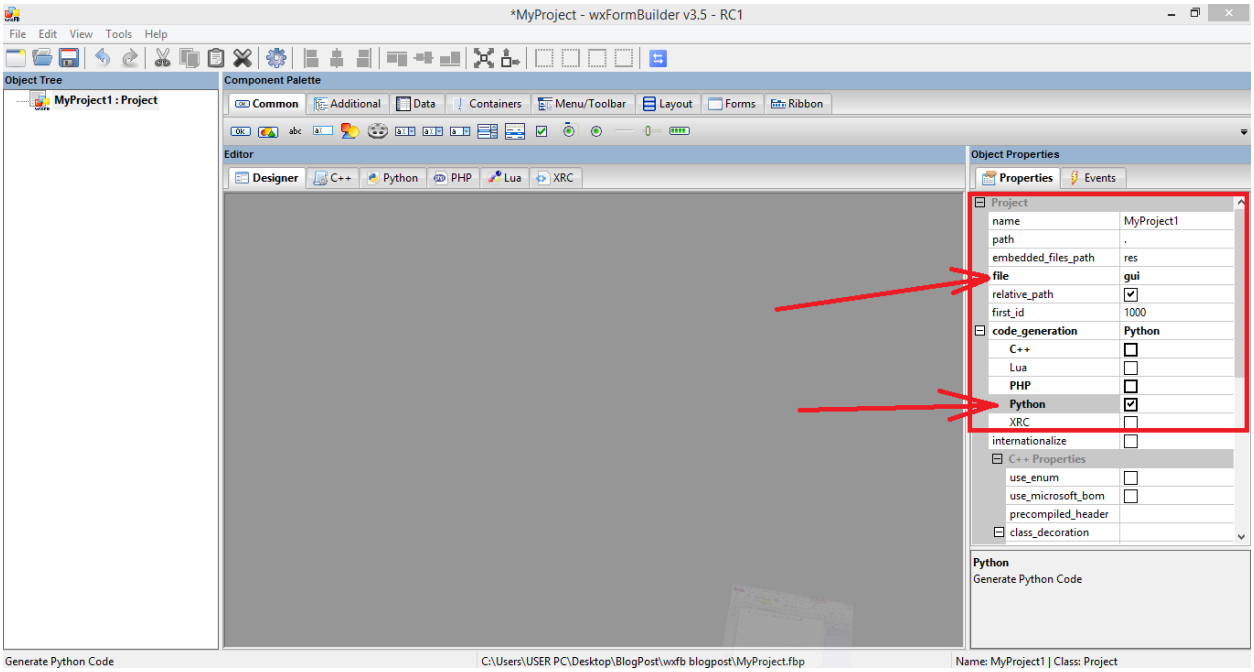
The prerequisite packages most be installed as above in the order mentioned and we should be good to go.

Building the GUI

Start wxformbuilder, by default a new project is already opened.

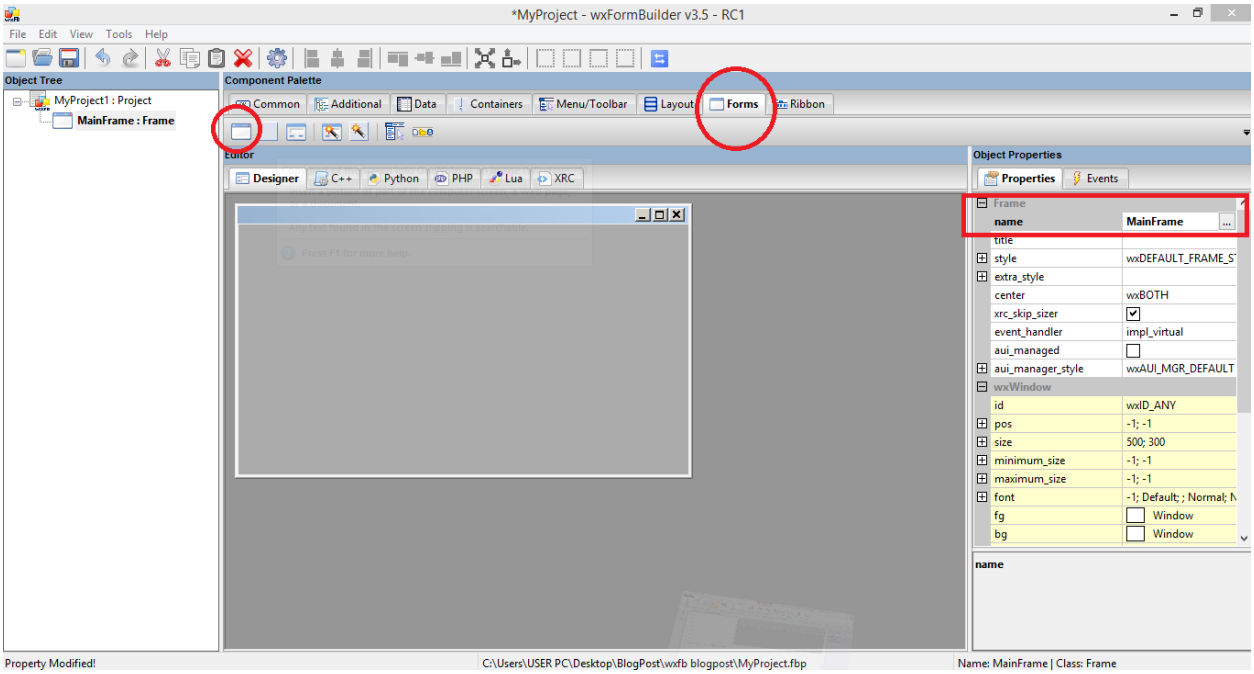
Save the new project to your desired directory/folder location on your PC with any name (I used: MyProject.fbp).

Set file name to ‘gui’ (or whatever name you want) and set code generation to ‘Python’ (since we want our GUI to be in wxPython).



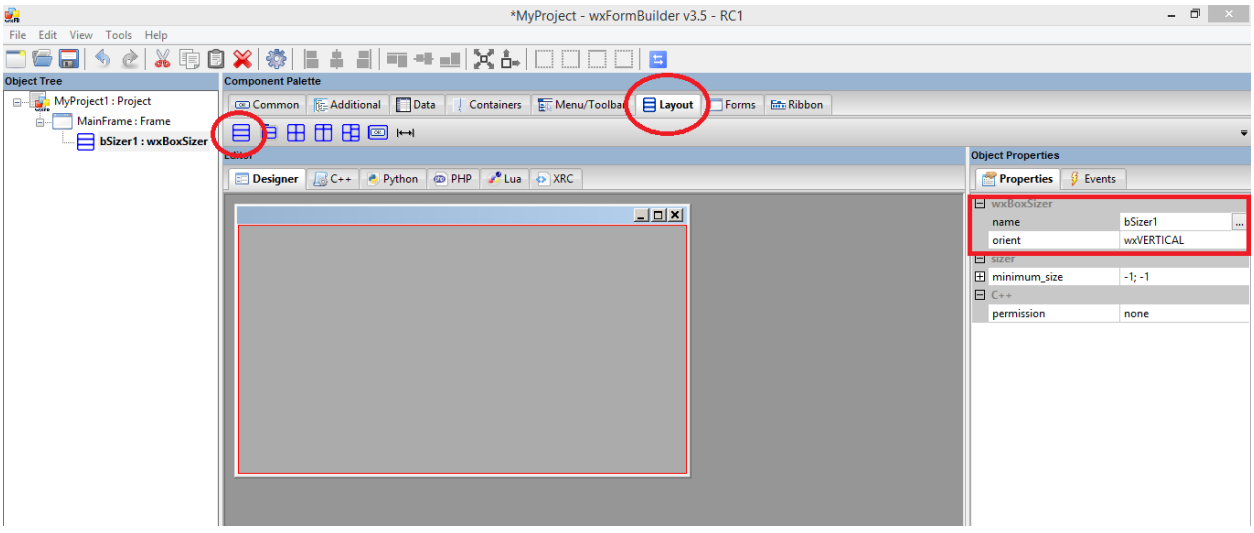
Add a frame from the “Form” tab.

Rename the frame name to "MainFrame" on the Object properties panel. This is optional, but for larger applications it will be necessary to differentiate between frames by their names.



Add a wxBoxSizer from the "Layout" tab.

Set "orient" to wxVERTICAL. This will hold our GUI components and stack them vertically



Add a static text (wxSTATICTEXT) from the “Common” tab:

From the tool bar, set the static text to “Expand” and “Align Center Horizontally”.

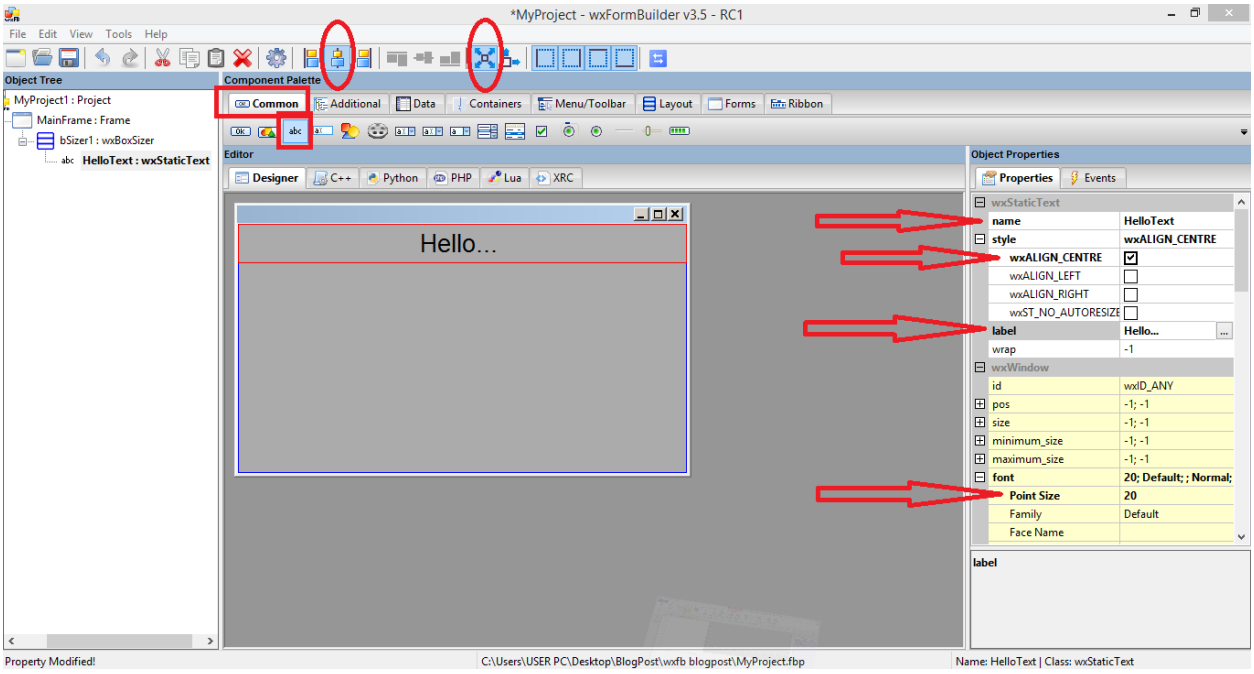
From the Object properties panel, set the following properties;

Name = HelloText

Style = wxALIGN_CENTRE

Label = Hello...

Font Point Size = 20



Add a drop down box (wxCHOICE) from the “Common” tab:

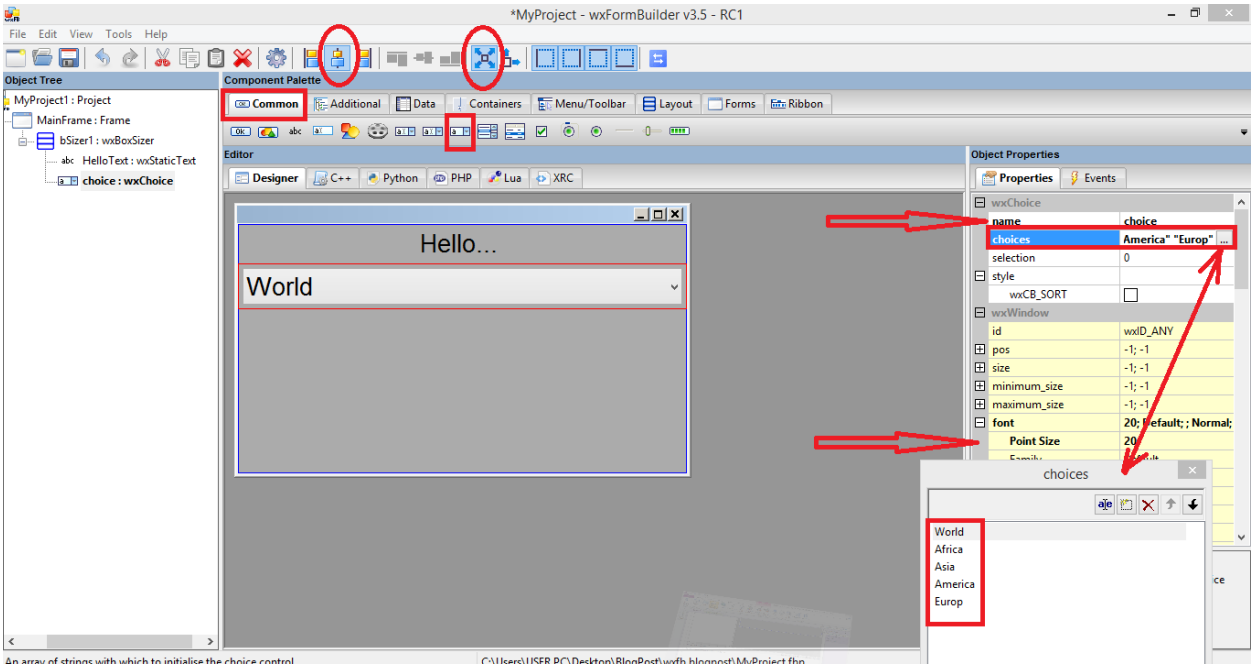
From the tool bar, set the drop down box to “Expand” and “Align Center Horizontally”.

From the Object properties panel, set the following properties;

Name = choice

Choices = World, Africa, America, Asia, Europ

Font Point Size = 20



Add a button (wxBUTTON) from the "Common" tab:

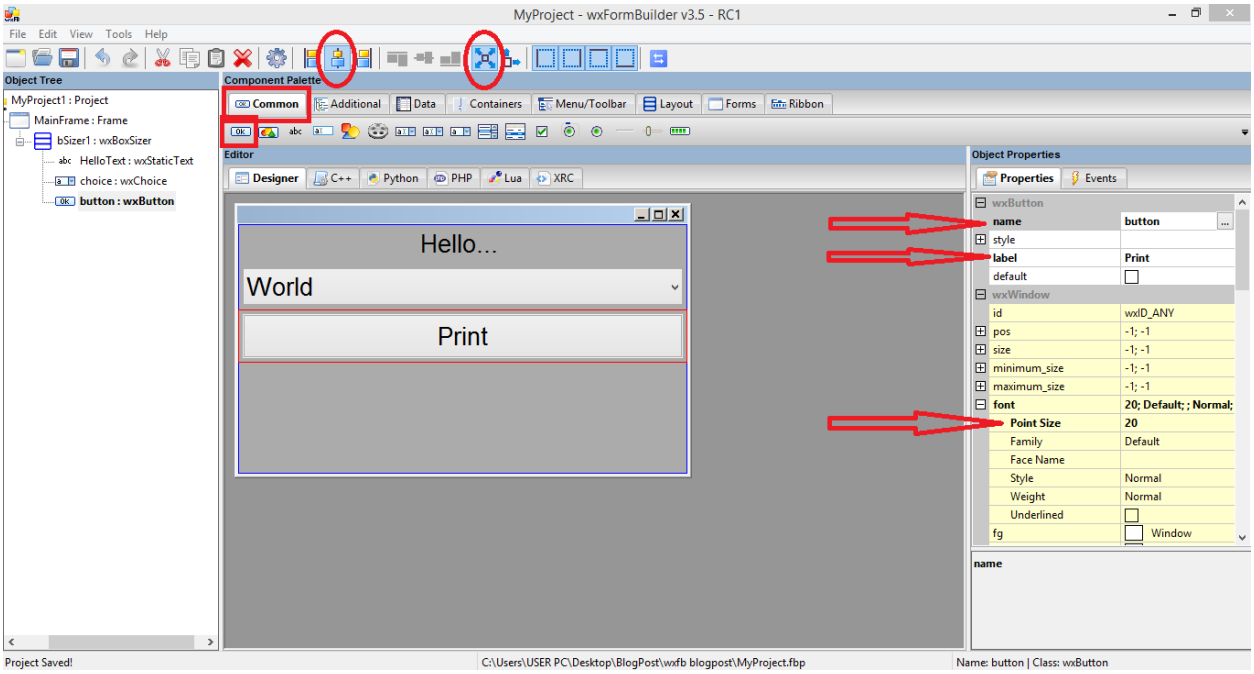
From the tool bar, set the drop down box to "Expand" and "Align Center Horizontally".

From the Object properties panel, set the following properties;

Name = button

Label = Print

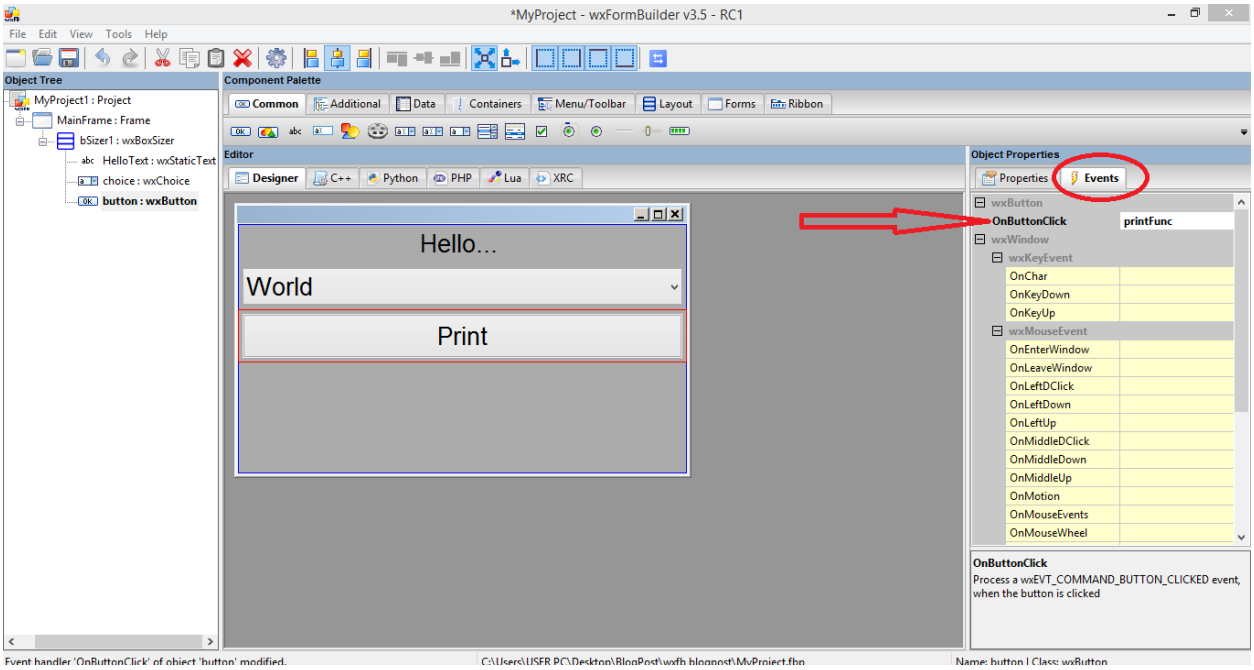
Font Point Size = 20



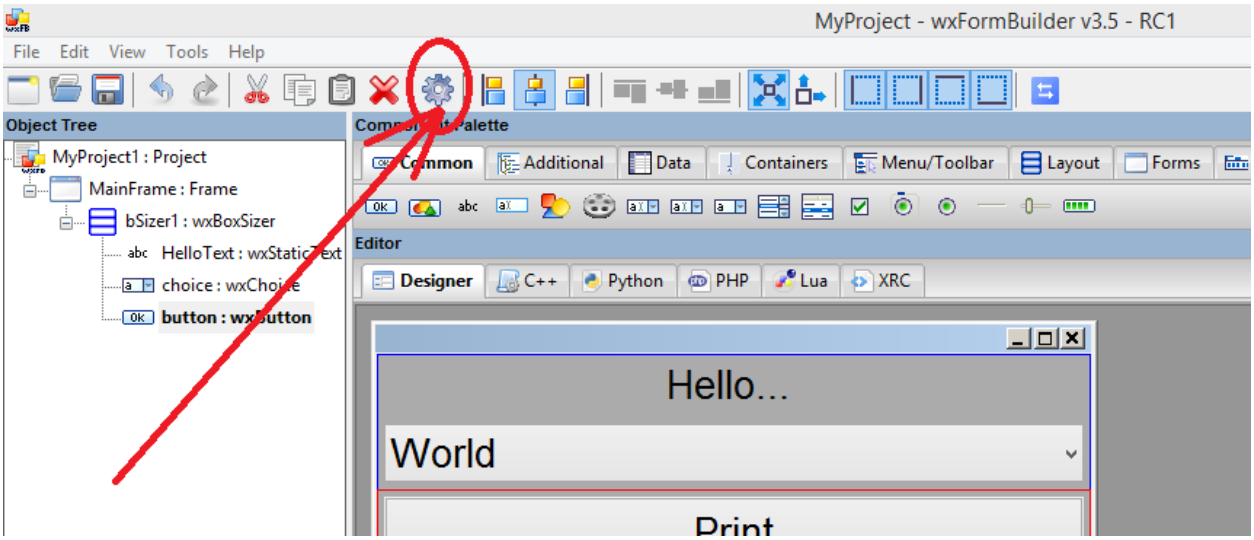
Bind an event for button clicks since we want to take selected text from the user and append/print the chosen text to a static text box above when the user clicks the button:

From the Object properties panel, select "Events" tab and set the OnButtonClick event to "printFunc" (o whatever name you want,

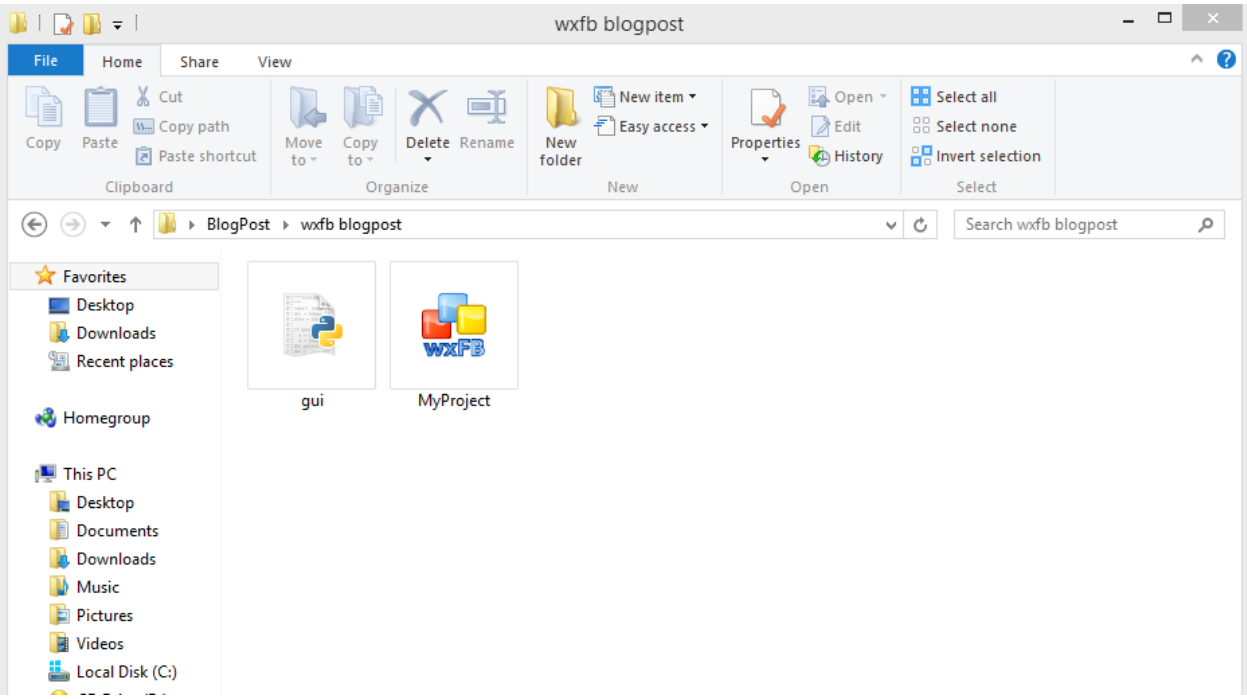
but be sure to remember the name as it is the method/function name that will control the button)



Save the wxformbuilder project to your project folder. Then click Generate code from the menu/tool bar or press F8 button on your keyboard.



The content of your project folder should contain the wxformbuilder project file and the generated file. You should end up with a file called "gui.py" in the directory you saved in. This file holds the code for generating the graphics.



Run the generated code "gui.py" as follow:-

Open the generated file "gui.py" in a text editor (I used sublime text editor)

Scroll to the bottom and add the code below;-

```
app = wx.App()
frame = MainFrame(None).Show()
app.MainLoop()
```

Above is mandatory in wxPython to create an application. It creates an object of MainFrame class and shows/displays it and starts the application on an infinite loop.

```
52
53      def __del__( self ):
54          pass
55
56
57      # Virtual event handlers, overide them in your derived class
58      def printFunc( self, event ):
59          event.Skip()
60
61
62
63  app = wx.App()
64  frame = MainFrame(None).Show()
65  app.MainLoop()
```

Now try running (Ctrl+B for Sublime text) the code and you should see something like this:



Now let's modify it to do something when user clicks the "Print" button. Recall that we bind an OnButtonClick event for the button named "printFunc" method/function.

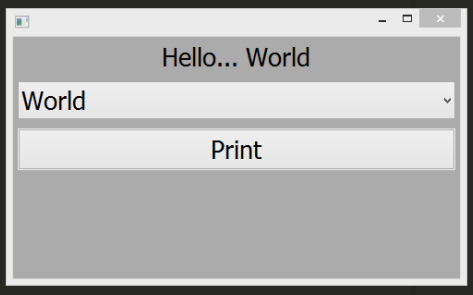
Edit the "printFunc" method/function to look like below;-

```
def printFunc( self, event ):
    hw = self.choice.GetStringSelection()
    self.HelloText.SetLabel('Hello... ' + hw)
    self.Layout()
```

The method/function gets the selected text and stores it in a variable "hw", then appends it to the static text label. The layout is retained by calling the layout method/function.

Recall the "*choice" and "HelloText" are names of the* "drop down box" and "static text" respectively as created earlier.

```
51          self.button.Bind( wx.EVT_BUTTON, self.printFunc )
52
53      def __del__( self ):
54          pass
55
56
57      # Virtual event handlers, overide them in your derived class
58      def printFunc( self, event ):
59          hw = self.choice.GetStringSelection()
60          self.HelloText.SetLabel('Hello... ' + hw)
61          self.Layout()
62
63
64
65
66  app = wx.App()
67  frame = MainFrame(None).Show()
68  app.MainLoop()
```



Building an executable:

Now we want to share the application with user who may not have python install on their computers. So we need a way to make our app to run on any computer without python installed on it. This process is generally referred to as "Freezing our Code".

The advantage of distributing this way is that your application will work even if the user doesn't already have the required version of Python installed. On Windows, and even on many Linux distributions and OSX versions, the right version of Python will not already be installed.

One disadvantage is that it will bloat your distribution by about 2MB. Another problem is that your application will not receive any security updates to the version of Python it uses unless you freeze a new version and get users to download it.

There are multiple tools to do this as compared in the table below, showing the Solutions and platforms/features supported:

http://docs.python-guide.org/en/latest/shipping/freezing/

Solution	Windows	Linux	OS X	Python 3	License	One-file mode	Zipfile import	Eggs	pkg_resources support
bbFreeze	yes	yes	yes	no	MIT	no	yes	yes	yes
py2exe	yes	no	no	yes	MIT	yes	yes	no	no
pyInstaller	yes	yes	yes	yes	GPL	yes	no	yes	no
cx_Freeze	yes	yes	yes	yes	PSF	no	yes	yes	no
py2app	no	no	yes	yes	MIT	no	yes	yes	yes

Installing PyInstaller

1- Run the following commands

pip install pyinstaller
Or
python.exe -m pip install pyinstaller

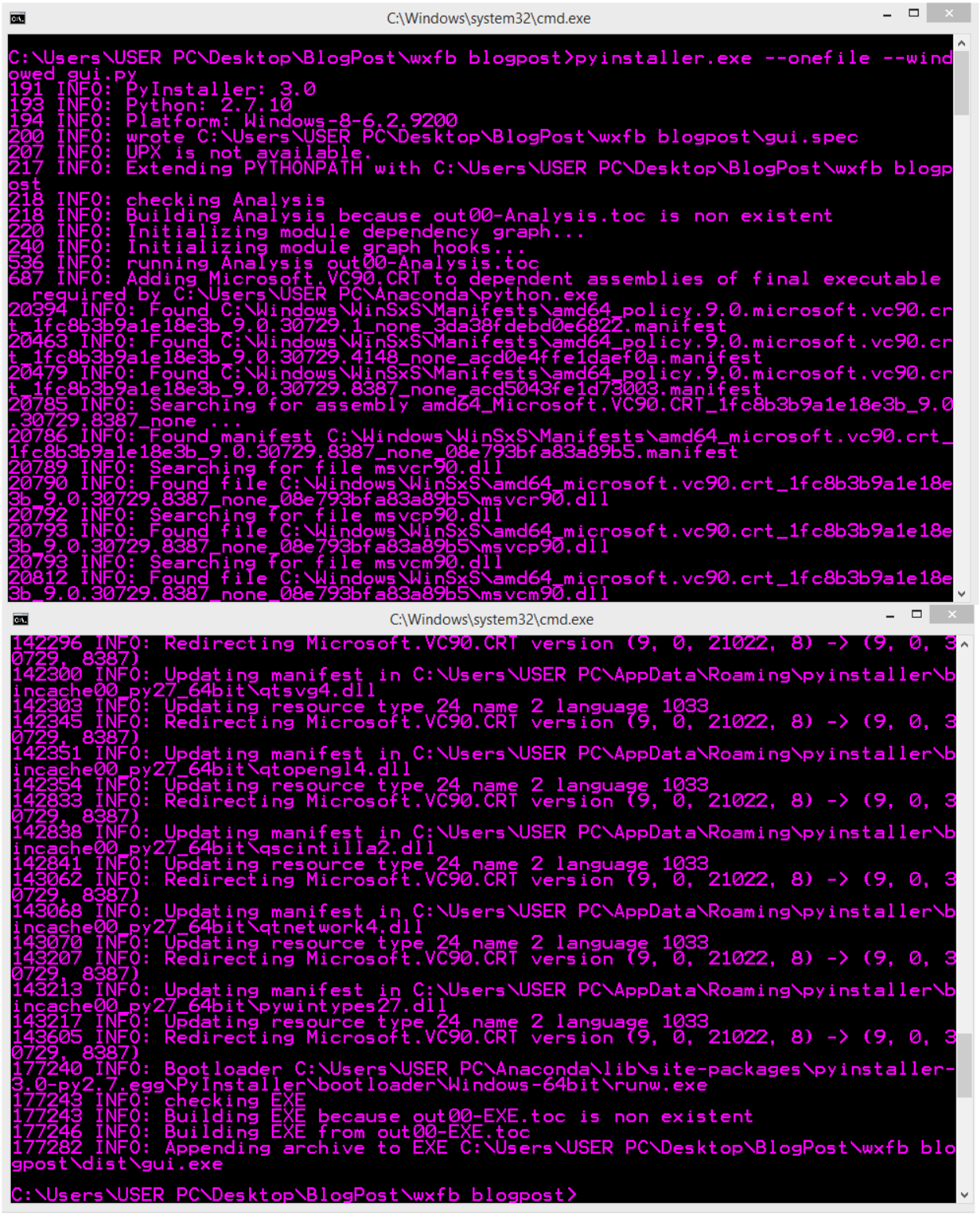
2- Download the PyInstaller package from and run this command from the extracted package directory

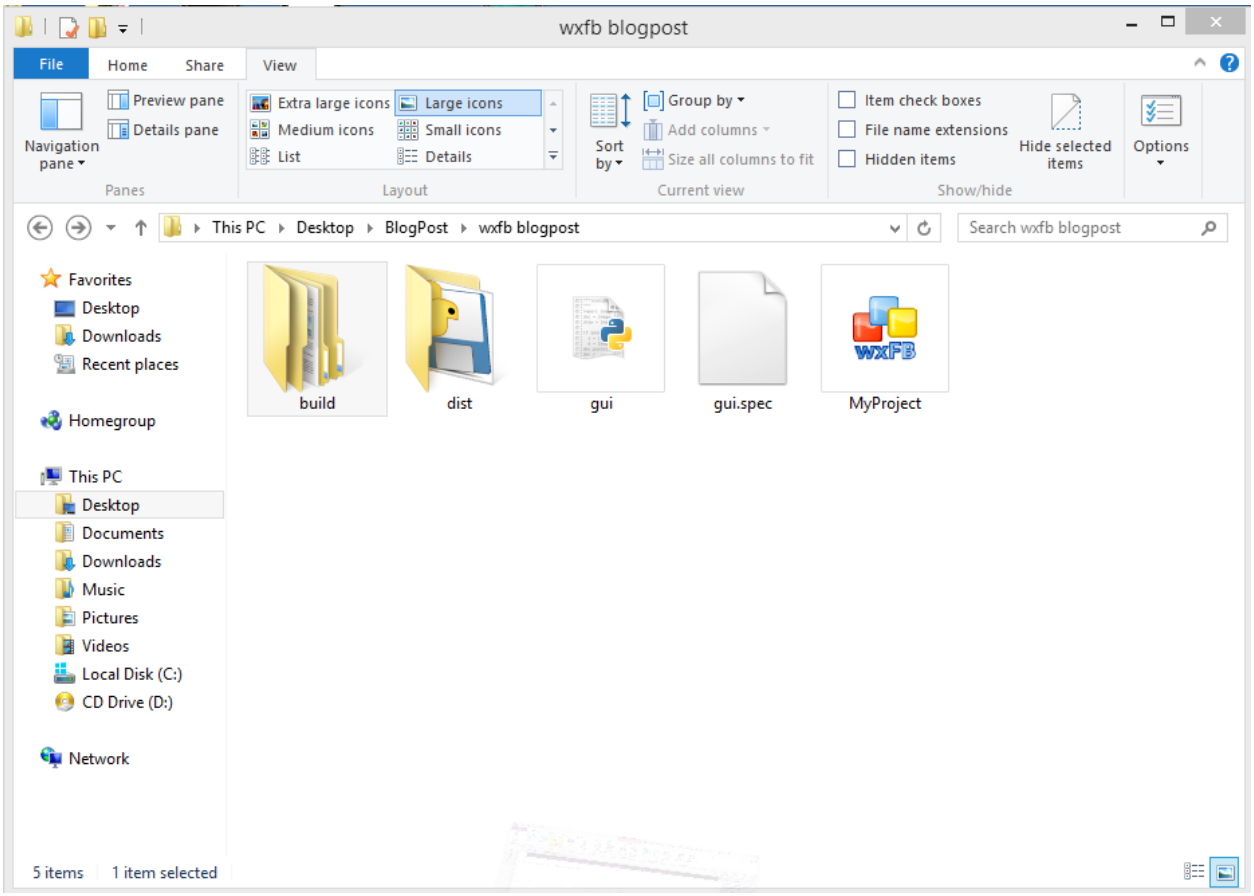
python setup.py install

Building the Executable:

From our application folder (directory), open cmd and run the command below;-

pyinstaller.exe --onefile --windowed gui.py


C:\Windows\system32\cmd.exe
C:\Users\USER PC\Desktop\BlogPost\wxfb blogpost>pyinstaller.exe --onefile --wind
owed gui.py
191 INFO: PyInstaller: 3.0
193 INFO: Python: 2.7.10
194 INFO: Platform: Windows-8-6.2.9200
200 INFO: wrote C:\Users\USER PC\Desktop\BlogPost\wxfb blogpost\gui.spec
207 INFO: UPX is not available.
217 INFO: Extending PYTHONPATH with C:\Users\USER PC\Desktop\BlogPost\wxfb blogp
ost
218 INFO: checking Analysis
218 INFO: Building Analysis because out00-Analysis.toc is non existent
220 INFO: Initializing module dependency graph...
240 INFO: Initializing module graph hooks...
536 INFO: running Analysis out00-Analysis.toc
687 INFO: Adding Microsoft.VC90.CRT to dependent assemblies of final executable
required by C:\Users\USER PC\Anaconda\python.exe
20394 INFO: Found C:\Windows\WinSxS\Manifests\amd64_policy.9.0.microsoft.vc90.cr
t_1fc8b3b9a1e18e3b_9.0.30729.1_none_3da38fdebd0e6822.manifest
20463 INFO: Found C:\Windows\WinSxS\Manifests\amd64_policy.9.0.microsoft.vc90.cr
t_1fc8b3b9a1e18e3b_9.0.30729.4148_none_acd0e4ffe1daef0a.manifest
20479 INFO: Found C:\Windows\WinSxS\Manifests\amd64_policy.9.0.microsoft.vc90.cr
t_1fc8b3b9a1e18e3b_9.0.30729.8387_none_acd5043fe1d73003.manifest
20785 INFO: Searching for assembly amd64_Microsoft.VC90.CRT_1fc8b3b9a1e18e3b_9.0
.30729.8387_none ...
20786 INFO: Found manifest C:\Windows\WinSxS\Manifests\amd64_microsoft.vc90.crt_
1fc8b3b9a1e18e3b_9.0.30729.8387_none_08e793bfa83a89b5.manifest
20789 INFO: Searching for file msvcr90.dll
20790 INFO: Found file C:\Windows\WinSxS\amd64_microsoft.vc90.crt_1fc8b3b9a1e18e
3b_9.0.30729.8387_none_08e793bfa83a89b5\msvcr90.dll
20792 INFO: Searching for file msvcp90.dll
20793 INFO: Found file C:\Windows\WinSxS\amd64_microsoft.vc90.crt_1fc8b3b9a1e18e
3b_9.0.30729.8387_none_08e793bfa83a89b5\msvcp90.dll
20793 INFO: Searching for file msvcm90.dll
20812 INFO: Found file C:\Windows\WinSxS\amd64_microsoft.vc90.crt_1fc8b3b9a1e18e
3b_9.0.30729.8387_none_08e793bfa83a89b5\msvcm90.dll
C:\Windows\system32\cmd.exe
142296 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
142300 INFO: Updating manifest in C:\Users\USER PC\AppData\Roaming\pyinstaller\b
incache00_py27_64bit\qtsvg4.dll
142303 INFO: Updating resource type 24 name 2 language 1033
142345 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
142351 INFO: Updating manifest in C:\Users\USER PC\AppData\Roaming\pyinstaller\b
incache00_py27_64bit\qtopengl4.dll
142354 INFO: Updating resource type 24 name 2 language 1033
142833 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
142838 INFO: Updating manifest in C:\Users\USER PC\AppData\Roaming\pyinstaller\b
incache00_py27_64bit\qscintilla2.dll
142841 INFO: Updating resource type 24 name 2 language 1033
143062 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
143068 INFO: Updating manifest in C:\Users\USER PC\AppData\Roaming\pyinstaller\b
incache00_py27_64bit\qtnetwork4.dll
143070 INFO: Updating resource type 24 name 2 language 1033
143207 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
143213 INFO: Updating manifest in C:\Users\USER PC\AppData\Roaming\pyinstaller\b
incache00_py27_64bit\pywintypes27.dll
143217 INFO: Updating resource type 24 name 2 language 1033
143605 INFO: Redirecting Microsoft.VC90.CRT version (9, 0, 21022, 8) -> (9, 0, 3
0729, 8387)
177240 INFO: Bootloader C:\Users\USER PC\Anaconda\lib\site-packages\pyinstaller-
3.0-py2.7.egg\PyInstaller\bootloader\Windows-64bit\runw.exe
177243 INFO: checking EXE
177243 INFO: Building EXE because out00-EXE.toc is non existent
177246 INFO: Building EXE from out00-EXE.toc
177282 INFO: Appending archive to EXE C:\Users\USER PC\Desktop\BlogPost\wxfb blo
gpost\dist\gui.exe
C:\Users\USER PC\Desktop\BlogPost\wxfb blogpost>



It’s that easy.

If the build was successful, the final executable, gui.exe, and any associated files, will be placed in the dist directory, which will be created if it doesn’t exist.

Let me briefly describe the options that are being used:

- --onefile is used to package everything into a single executable. If you do not specify this option, the libraries, etc. will be distributed as separate files alongside the main executable.
- --windowed prevents a console window from being displayed when the application is run. If you’re releasing a non-

graphical application (i.e. a console application), you do not need to use this option.

- gui.py the main source file of the application. The base name of this script will be used to name of the executable, however you may specify an alternative executable name using the --name option.

Check the PyInstaller Manual for more configuration information.

After the build, a gui.spec file will be created. This file contains all of the options used to run PyInstaller, and can be fed back into PyInstaller for future builds in place of the command line options, if desired.

To add icon and version text to the exe file, use command below (version.txt will be provided, see the manual);-

pyinstaller.exe --onefile --windowed --icon=src\img\favicon.ico --version-file=version.txt gui.py

Building the Installer:

PyInstaller merely assembles the files needed to run your python program; it is not an installer builder. An installer will further compress the app size.

There are plenty of good installer builders out there including some that are open source e.g.: NSIS and Inno Setup.

Download the code:

The files generated throughout the tutorial can be downloaded from the link below. Size is 25mb

https://www.dropbox.com/s/w2e6g7mercdqly0/HelloGUI.zip?dl=0

Exercise:

As an exercise;

1- Add an icon and a title to the title bar?
2- Set a color to the frame's background?
3- Add two buttons to exit and reset the app?
4- Finally, add menus to handle the button tasks and to provide help for the app?

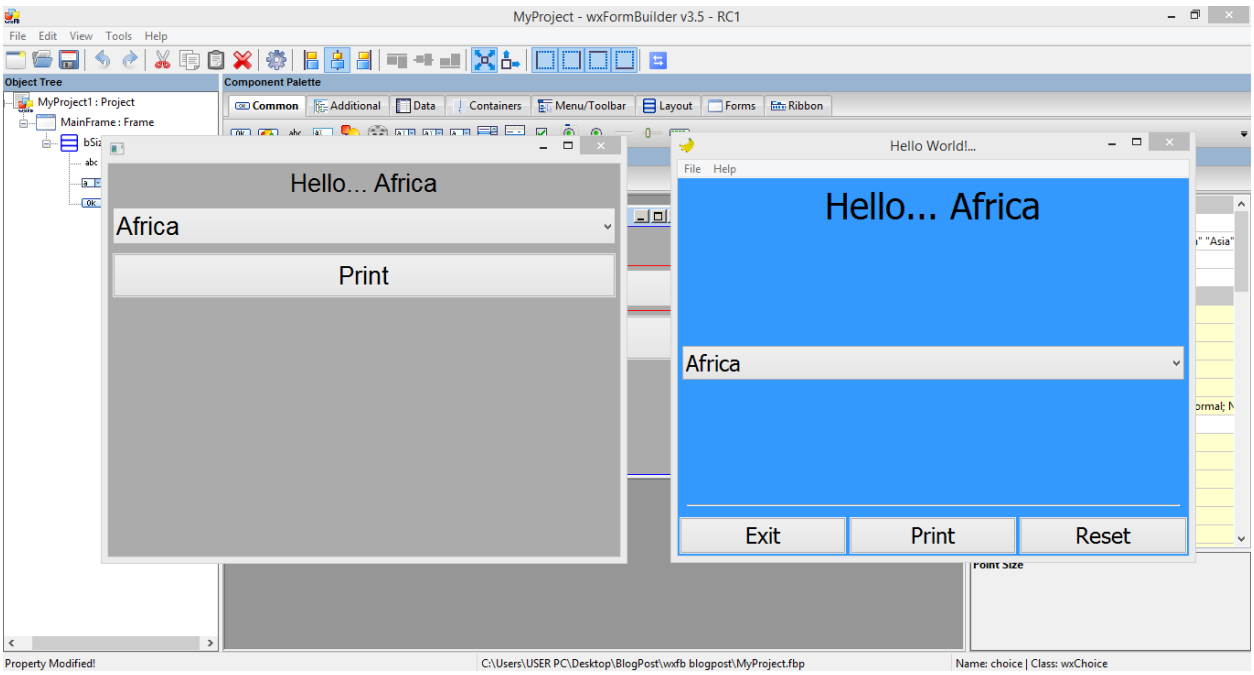
See final result to the exercise in image below.

Solution to this and more are available in video format on this link: **www.UmarYusuf.com/wxpy**



Good luck, have fun, and I hope my tutorial was useful to you?

Thanks for reading!
Email: umaryusuf49@gmail.com
Website: www.UmarYusuf.com



Referenced Materials

Other tutorials that motivated this post

1- Ultra Quick GUIs with Wxformbuilder/Python by Vighnesh Birodkar
https://www.facebook.com/vighnesh.birodkar

https://vcansimplify.wordpress.com/2013/04/08/ultra-quick-guis-with-wxformbuilderpython/

2- Rapidly building a python GUI application by Sebastian Nilsson
http://sebastiannilsson.com/en/blogg/rapidly-building-python-gui-application/

3- How to Install PyQt5 and Build Your First GUI in Python 3.4
http://projects.skylogic.ca/blog/how-to-install-pyqt5-and-build-your-first-gui-in-python-3-4/

4- Tutorial: Creating GUI Applications in Python with QT by Alex Fedosov
http://www.cs.usfca.edu/~afedosov/qttut/

5- Creating an Executable from a Python Script by Matt Borgerson
https://mborgerson.com/creating-an-executable-from-a-python-script